

Lesson 3.3 — Rejecting AI Output

Lesson 3.3 — Rejecting AI Output

Engineers who can reject AI output cleanly are worth more than engineers who only know how to accept it. This lesson teaches you both the discipline and how to talk about it.

By the end of this lesson:

- You will know the five legitimate reasons to reject a diff.
 - You will know the four signals that the prompt — not the model — is the problem.
 - You will have language for talking about rejection that does not sound negative.
 - You will have a 60-second answer to “How do you review AI-generated code?”
-

1. Why this is the hardest part

Rejecting AI output goes against the dopamine loop. You wrote a prompt. The AI produced something. The output looks plausible. The path of least resistance is **accept**. Senior engineers fight that path every day.

The skill is not “be a perfectionist.” The skill is **calibrated suspicion** — the experienced sense that this part of an AI diff is fine and that part is wrong, even when both parts compile.

Interviewers know this. The question — direct or implied — is always present:

“Show me you can tell the difference between AI output you should accept and AI output you should reject.”

This lesson is the answer.

2. The five legitimate reasons to reject a diff

2.1. The diff does the wrong thing

The most basic reason. The output works, but it is not what you asked for. The model interpreted your prompt differently than you intended.

"I asked for a debounced search. The diff implemented a throttled search. They look similar; they are not the same. Reject."

2.2. The diff does the right thing wrongly

The output is functionally correct but uses an approach inappropriate for your codebase: deprecated APIs, the wrong abstraction, fighting your existing patterns.

"The diff used `getServerSideProps` in a Next.js 14 App Router project. The right behavior, the wrong era. Reject."

2.3. The diff is too big

The output sprawls into files you did not ask about. Even if every line is correct, the change is unreviewable. **Force a smaller diff.**

"I asked to add a prop. The diff touched eleven files including a 'while we are here' refactor of the parent. Reject and re-prompt with constraints."

2.4. The diff hides complexity

The output works but introduces an abstraction (a wrapper, a context, a generic) you did not ask for and that adds more cognitive cost than it saves.

"The diff wrapped my hook in a context provider 'for future flexibility.' We do not have a use case for that flexibility. Reject — over-engineering."

2.5. You cannot explain a part of it

The output works and looks reasonable, but there is at least one block you cannot walk through line by line. **If you cannot explain it, you cannot ship it.**

"The diff used a one-line ternary with array spread that I had to stare at for two minutes. Reject — even if the spread is correct, code I cannot read in three seconds is code I will misread under pressure."

3. The four signals that the prompt is the problem

When the AI gives you bad output repeatedly, it is rarely the model's fault. It is your prompt. Look for these signals:

3.1. You keep adding "but also..." in follow-ups

"Add a loading state... but also do not change the styling... but also use the existing Spinner... but also..."

Each "but also" is a constraint that should have been in the original prompt. **Stop and rewrite the prompt with those constraints up front.**

3.2. The output keeps drifting in the same wrong direction

If three iterations all produce variants of the wrong thing, your prompt is anchoring the model toward that wrong thing. Often a single misleading word is the culprit. *"Make this more efficient"* often produces premature optimization. *"Make this cleaner"* often produces unnecessary abstractions. **Audit your verbs.**

3.3. You find yourself debugging the prompt instead of the code

If you have spent more than two minutes deciding whether to reject a diff, the prompt was probably under-specified. **Reject the diff, rewrite the prompt, run again.** Faster than wrestling with the output.

3.4. The output is plausible but addresses a different problem

The most insidious failure. The diff compiles, runs, and looks good — but it solves a problem adjacent to yours. This means your prompt was ambiguous about the problem itself. Go back and write the problem statement more precisely.

4. The senior way to reject

How you communicate rejection — to yourself, to teammates, in interviews — matters as much as the rejection itself.

4.1. Reject the output, not the tool

Bad: *"The AI is useless for this kind of work."*

Better: *"The output did not fit our codebase. The next prompt will pin the codebase patterns more explicitly."*

The first version sounds like a bias. The second sounds like an engineer.

4.2. Reject in writing, briefly

When you reject a diff, write one line about why. To yourself. To the PR description if relevant. *"Rejected because it changed the public API of useScores; constraint added to next prompt."* This builds your personal failure-mode catalog **and** signals the discipline to anyone reading your history.

4.3. Reject without rage

Frustration is contagious. If you find yourself reflexively dismissing AI output, your prompts will get worse and your suspicion will get less calibrated. Maintain a working assumption that the model is **trying** to be helpful and that bad output is fixable. That neutrality is what produces clean prompts.

5. The "show me a rejection" interview question

Increasingly common in 2026 interviews:

"Show me a recent diff from your AI tool that you rejected. Walk me through why."

Have an answer. Use this structure:

"Recent example. I asked the AI to extract a sub-component from a large file. The diff did extract a component, but it also helpfully refactored the parent's

state into a useReducer. The state machine had three transitions and lived in one place – useReducer was over-engineered for it. I rejected the diff and re-prompted with: ‘extract only the sub-component; do not change state shape or hooks in the parent.’ Second pass was clean and shipped.”

That answer hits:

- **Specific situation** (extract sub-component).
- **Specific over-reach** (state refactor I did not ask for).
- **Specific rejection reason** (over-engineering, scope creep).
- **Specific re-prompt** (constraint added).
- **Resolution** (shipped).

Six sentences. ~30 seconds. **Drill it.**

6. The 60-second AI code review answer

The flagship version of this skill in interview form:

“My AI code review is different from my human code review. With human code, I assume good intent and read for design choices. With AI code, I read more skeptically because the failure modes are different – confident-looking calls to APIs that do not exist, plausible-looking patterns that conflict with our codebase, and ‘helpful’ refactors of code I did not ask about. My AI checklist has four items: does the diff do exactly what I asked, no more; does every API call point to a real, current API; does the change respect our codebase’s existing patterns; and can I explain every line in three seconds. If any answer is no, the diff goes back.”

That is ~110 words, ~50 seconds. It is a complete, defensible answer to the most common AI-code-review question. **Memorize it word for word.**

7. The “fabrication” / hallucination question

Interviewers love to ask:

“How do you handle AI hallucinations?”

The wrong answer: *“I check things on Stack Overflow.”* That is junior.

The right answer:

“Three layers. First, type-checking and the compiler catch most fabricated APIs immediately – the pain of a broken type is your first line of defense. Second, my eye is calibrated for the most common fabrication patterns: invented method names, parameter orders that should be alphabetical, and ‘utility’ libraries that do not exist. Third, when I am genuinely uncertain, I pull the actual library docs in another window and confirm. The defense is layered – none of those alone is enough, but the combination catches almost everything.”

That answer:

- Names three concrete defenses.
- Treats type-checking as the first defense (a senior signal).
- Mentions calibrated suspicion (Lesson 3.3 vocabulary).
- Closes with humility – pulling docs to verify.

Drill this too.

8. The “AI sometimes gives wrong code” trap

Sometimes the interviewer leads with this:

“AI sometimes gives wrong code. Doesn’t that worry you?”

This sounds like an attack. It is not. It is an opening. **Take it.**

“Sometimes is the right word. The work is the calibration – knowing which sometimes. After a year of doing this daily, I have a feel for the patterns where AI is reliable and the patterns where it is dangerous. Reliable: type derivation, boilerplate, conversion between formats, scaffolding. Dangerous: cross-module reasoning, anything to do with money, anything where the test would not catch the failure mode. The skill is not avoiding ‘wrong code’; it is knowing where to expect it.”

The trap was: ‘doesn’t that worry you’ invites you to defend the AI. **Don’t defend the AI.** Defend your calibration. The AI is a tool; tools have failure modes; engineers know their tools’ failure modes.

9. Practice exercise – the rejection journal

For two weeks, every time you reject an AI diff, log:

DATE: _____

PROMPT: _____

OUTPUT: _____

REJECTED BECAUSE: (pick one or more)

☐ wrong thing

☐ right thing wrongly

☐ too big

☐ hides complexity

☐ cannot explain

RE-PROMPT (if you wrote one):

LESSON:

After two weeks you will have:

- A pattern library of your most common rejection reasons.
- A list of words/phrases in your prompts that consistently produce bad output.
- A vocabulary you can use in interviews (“I have a journal of rejected diffs and the most common reason is...” – which is **astonishingly senior signaling**).

10. The “I do not reject” red flag

If reading this lesson made you realize you almost never reject AI output, that is the lesson.

Engineers who do not reject are not reviewing. Try this for one week:

- Force yourself to reject one out of every five AI diffs you would have accepted.
- Reject because of any of the five reasons in section 2 – even if you would normally let it slide.
- Re-prompt and compare.

You will find: the second pass is usually noticeably better. The standard you held for the first pass was lower than the standard the model could meet with a slightly better prompt. This exercise alone, repeated, separates juniors who use AI from seniors who use AI.

11. Common pitfalls

11.1. Rejecting performatively in interviews

Saying *“I reject AI code all the time”* without specifics. The interviewer asks for an example, you do not have one, and you sound rehearsed. **Always have a real example.**

11.2. Rejecting reactively, not deliberately

Rejecting because the diff “feels off” without articulating why. That feeling is data, but it has to translate to a sentence. Otherwise you cannot prompt better next time.

11.3. Confusing rejection with failure

A rejected diff is not a failure. It is the **review step working as intended**. If your tool produces a wrong diff and you catch it, the system worked. The failure case is the wrong diff that ships.

11.4. Rejecting because of taste alone

Sometimes you reject because the diff is “ugly.” That is allowed, but be honest about it. Style preferences are not architectural concerns. **Distinguish them when explaining your rejection.**

12. CHECK YOURSELF

- ☐ Can you list the five legitimate reasons to reject a diff?
- ☐ Can you list the four signals that your prompt — not the model — is wrong?
- ☐ Do you have a real example of a recent rejection you can describe in 30 seconds?
- ☐ Can you deliver the section-6 AI code review answer in under 60 seconds, no notes?

- ☐ Can you handle the section-8 trap question without defending the tool?
-

13. Where you are now

You have:

- A vocabulary for AI-assisted coding work (Chapter 1).
- A defended answer to “why are humans still needed” (Chapter 2).
- A library of patterns plus the discipline to deploy them (Chapter 3.1, 3.2).
- A rejection skill plus the language to communicate it (this lesson).

That is the conceptual half of the course. Chapter 4 takes us into the **practical** interview formats: live coding, debugging AI-written bugs, and reviewing AI output in real time.

Open [../chapter-04-live-scenarios/README.md](#).